

Exercicio 1: Configuración da máquina virtual

Comezaremos importando e **configurando a nosa máquina virtual**. En tódalas máquinas en VirtualBox teremos un adaptador de rede **NAT**, que nos proporcionará acceso a rede, e outro adaptador de rede **Adaptador Solo Anfitrión** que nos permitirá conectarnos a esta a través de SSH dende o noso equipo.

1. En VirtualBox vai a **Archivo > Herramientas > Administrador de Red**.
2. Preme en **Crear**.
3. Fai dobre *click* encima da rede que se creou. Preme en **Adaptador DHCP** e activa **Habilitar servidor** e modifica a seguinte configuración:
 - **Limite inferior de direcciones**: 192.168.56.100
 - **Limite superior de direcciones**: 192.168.56.200
1. Preme en **Aplicar**.
2. Importa a OVA en VirtualBox. Cámbialle o nome da máquina en VirtualBox por **dapw_iniciais_server_01**.
3. Engade un novo Adaptador de rede a máquina na configuración desta en VirtualBox (conserva o adaptador **NAT** para ter acceso a rede). Neste novo adaptador utilizaremos o **Adaptador Solo Anfitrión** e a rede creada nos pasos anteriores.
4. Arranca a máquina e inicia sesión co usuario **dadmin**. Executa o comando **ip a** para consultar a IP. **Realiza unha captura desta IP**. É importante que anotes esta IP porque a necesitaremos para acceder por SSH dende o equipo anfitrión.

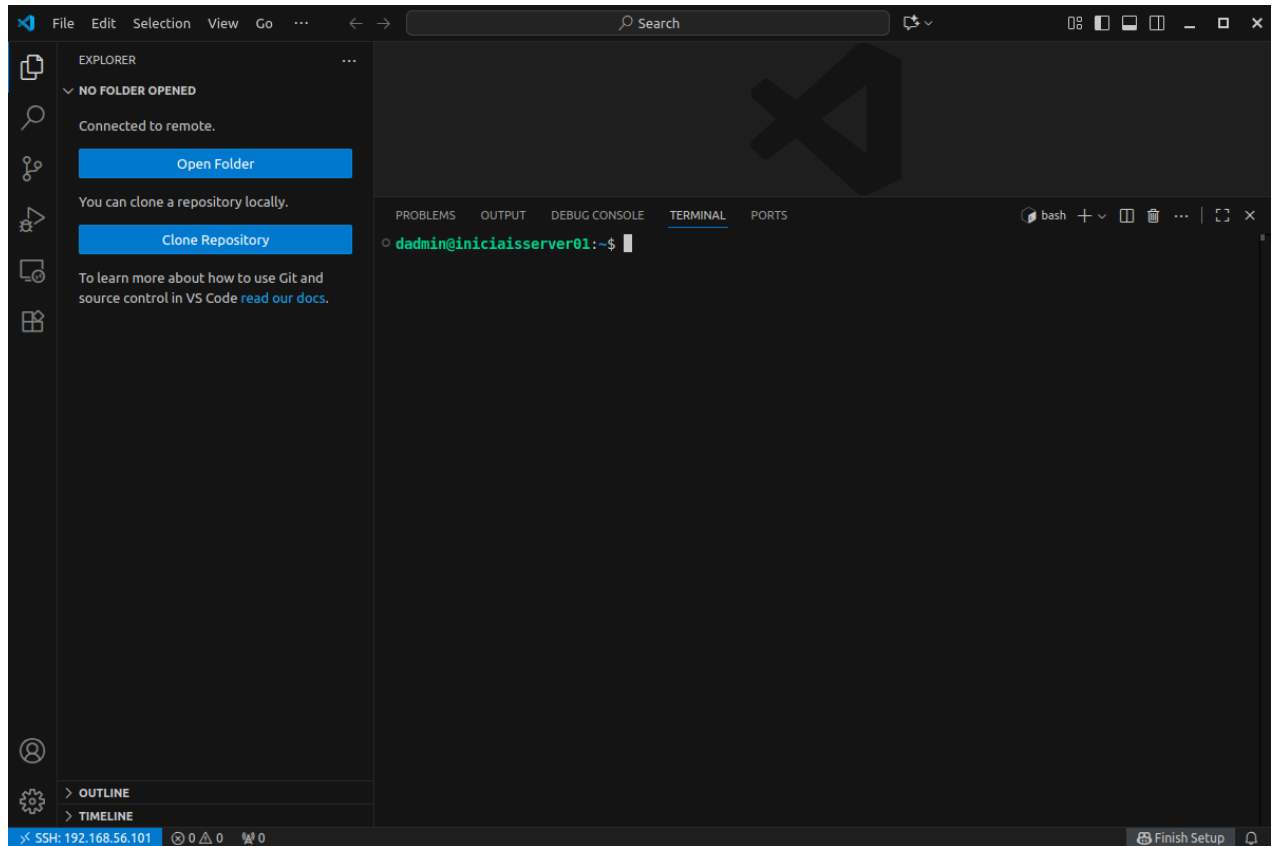
```
dadmin@iniciaisserver01:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:68:94:f2 brd ff:ff:ff:ff:ff:ff
    inet6 fd17:625c:f037:2:a00:27ff:fe68:94f2/64 scope global dynamic mngtmpaddr
        valid_lft 86397sec preferred_lft 14397sec
    inet6 fe80::a00:27ff:fe68:94f2/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:77:ea:59 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic enp0s8
        valid_lft 596sec preferred_lft 596sec
    inet6 fe80::a00:27ff:fe77:ea59/64 scope link
        valid_lft forever preferred_lft forever
```

5. Modifica o nome interno da máquina por **iniciaisserver01**, para iso executa os seguintes comandos que reiniciarán a máquina co novo nome:

Unha vez configurada a máquina, **procederemos a conectarnos por SSH**. Isto poderíámolo facer dende un terminal calquera ou nalgún *software* específico. No noso caso utilizaremos unha **extensión de Visual Studio Code** que facilita tanto o acceso a máquina como a edición de ficheiros.

1. Instala en Visual Studio Code a extensión **Remote - SSH**.
2. Preme **F1** e selecciona a opción **Remote - SSH > Add New SSH Host**. Esta opción permitiranos engadir a nosa máquina a lista de máquinas a que nos podemos conectar.
3. Na opción do comando SSH escribe o seguinte comando **ssh dadmin@IP_MÁQUINA**. Na hora de elixir o arquivo de configuración de SSH, elixe o do teu usuario.

4. Agora vamos a conectarnos a máquina, preme `F1` e selecciona `Remote - SSH > Connect to Host`. Selecciona a IP da máquina virtual. Abrirase unha nova ventá de VSC. Preme en `Continuar` e pon o contrasinal do usuario `dadmin`.
5. Abre un novo terminal. Verás que o terminal (`Terminal > Nuevo Terminal`) se inicia na máquina virtual (fíxate no nome da máquina). **Realiza unha captura.**



Unha **snapshot en VirtualBox** é unha “foto” do estado completo dunha máquina virtual (disco, memoria e configuración) nun momento dado. Serve para **volver atrás doadamente**. se algo falla (por exemplo, tras instalar software, facer probas ou actualizar o sistema). Deste xeito, poderemos volver a este estado inicial sen software instalado pero coa máquina configurada para realizar novas tarefas sen ter que realizar estes pasos previos.

1. Por último é **MOI IMPORTANTE**, desconéctate da máquina en Visual Studio Code e apaga a máquina virtual. **Crea unha snapshot en VirtualBox** de nome `admin_remota_configurada`.

Exercicio 2: Xestor de paquetes Composer

Xa temos preparada e configurada a nosa máquina para realizar a conexión remota. Agora comezaremos a tarefa propiamente dita. Para comezar, **instalaremos PHP e o xestor de paquetes Composer**.

1. Prende a máquina e volve a conectarte vía SSH.
2. Actualiza o equipo. É recomendable ter o sistema operativo actualizado por seguridade. Utiliza os seguintes comandos:
 1. Instala PHP. **Realiza unha captura da versión instalada.**
 2. Instala `Composer`. **Realiza unha captura da versión instalada.**

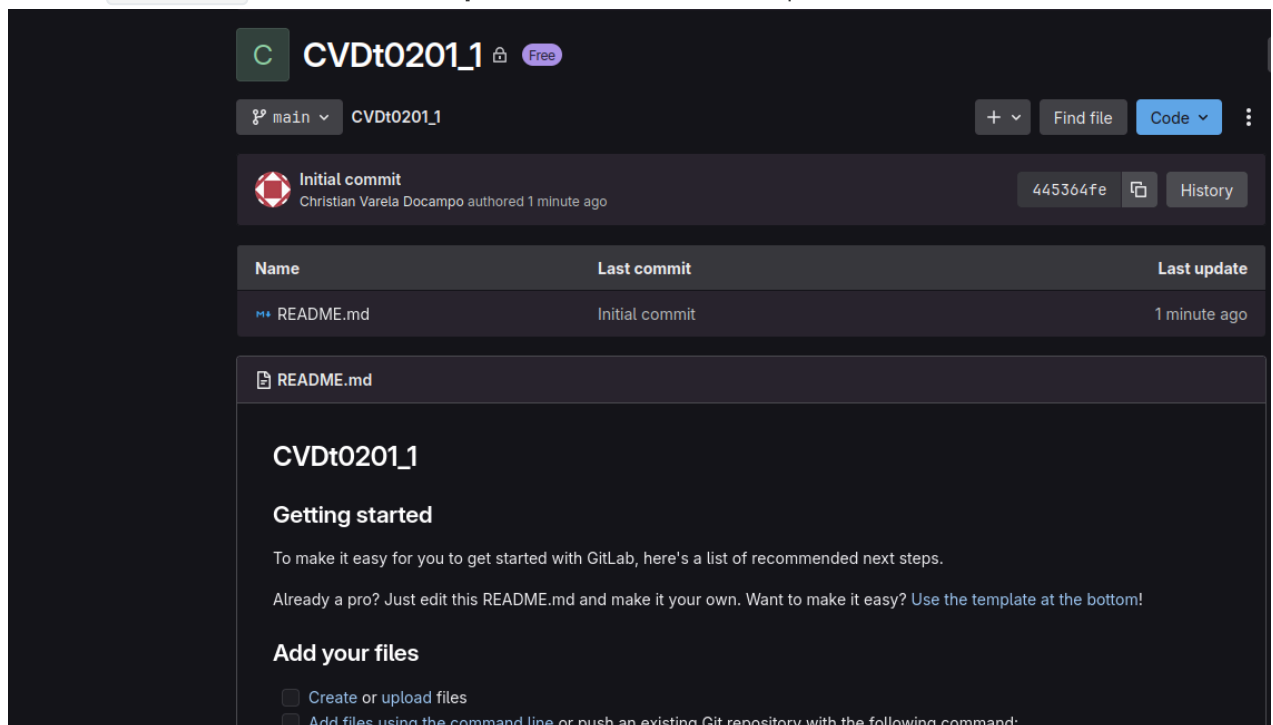
```

• dadmin@iniciaisserver01:~/remoto$ php --version
PHP 8.2.29 (cli) (built: Jul  3 2025 16:16:05) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.2.29, Copyright (c) Zend Technologies
    with Zend OPcache v8.2.29, Copyright (c), by Zend Technologies
• dadmin@iniciaisserver01:~/remoto$ composer --version
Composer version 2.5.5 2023-03-21 11:50:05

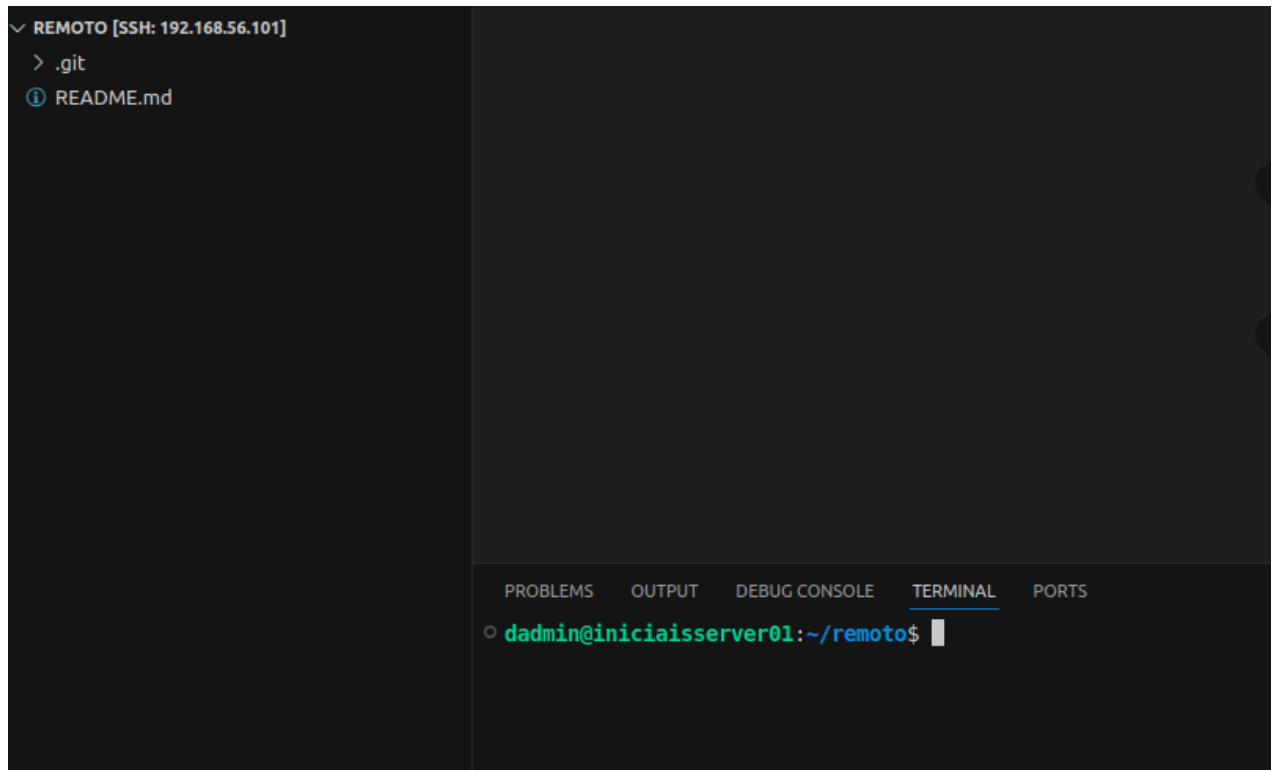
```

Agora crearemos unha conta en GitLab.com co correo electrónico do IES San Clemente. Se inicias mediante Google, terás que ir a opcións para poñer un contrasinal para poder clonar os repositorios. Ademais **crearemos un repositorio novo** en GitLab.com e **clonarémo-lo na nosa máquina**. Se xa tes a conta creada xa non terás que creala e poderás saltarte os primeiros pasos.

1. Crea unha conta en GitLab.com.
2. Indica o contrasinal en preferencias se e que iniciaches mediante Google.
3. Crea un novo proxecto (en GitLab chámanlle proxectos ao repositorios) de nome `iniciaist0201_1` con ficheiro `Readme.md`. **Realiza unha captura** da interface web co proxecto en GitLab.



4. Clona o repositorio na máquina virtual.
5. Preme na icona de ficheiros do menú da esquerda de Visual Studio Code. Preme en `Abrir Carpeta`. Selecciona o directorio onde clonaches o repositorio. Acepta tódalas mensaxes. **Realiza unha captura** da interface de VSC onde se vexan os ficheiros do repositorio e a terminal.

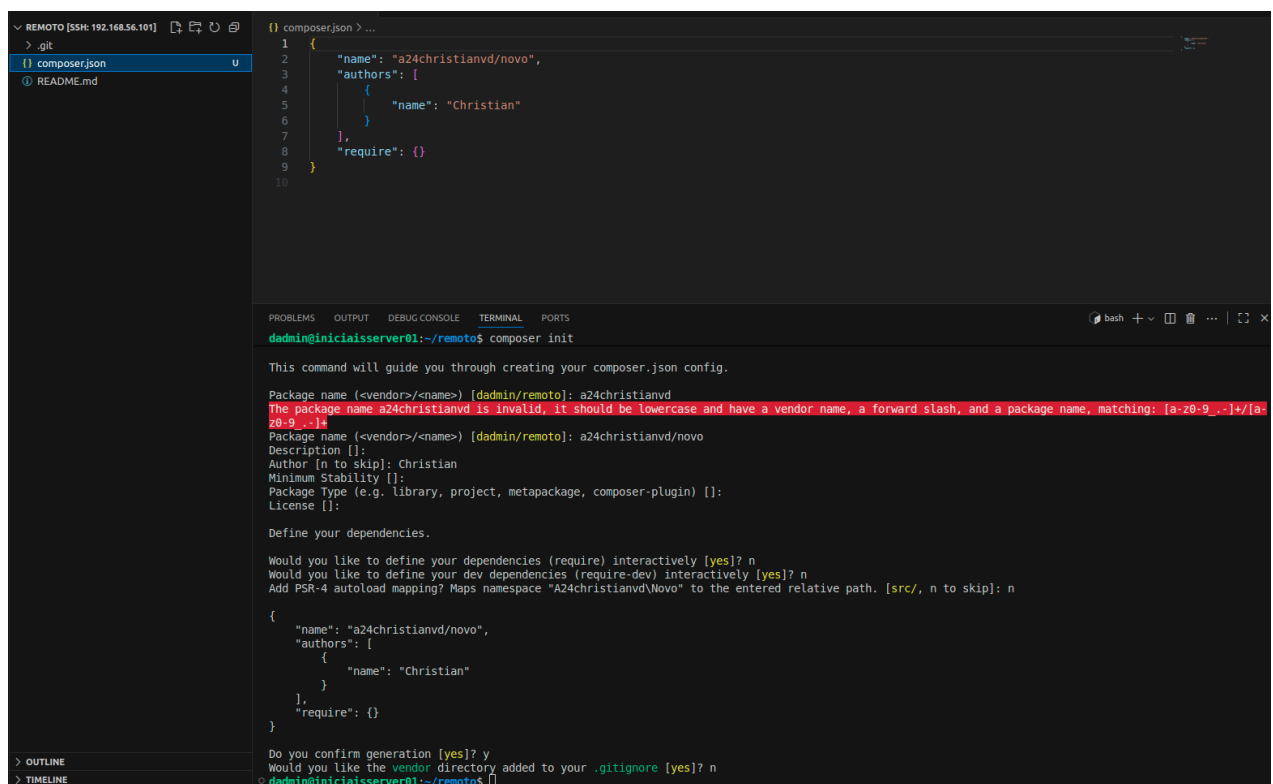


A continuación comenzaremos a **traballar co xestor de paquetes Composer**. Veremos como se instalan paquetes utilizando esta ferramenta.

1. Executa o comando `composer init`. Ten en conta:

- Nas preguntas que se piden datos, tan só tes que poñer o teu nome na pregunta do autor.
- Nas de contestar `yes/no` selecciona todas `no` agás na de que estás seguro de crear o ficheiro `composer.json`.

1. Abre o ficheiro `composer.json` e **realiza unha captura** do contido deste.



2. Instala o pacote `fakerphp/faker` com `Composer`.
3. Abre o ficheiro `composer.json` e `composer.lock` e realiza capturas do contido destes ficheiros.

```
{ } composer.json U X
{ } composer.json > { } require > [add] fakerphp/faker
1  {
2      "name": "a24christianvd/novo",
3      "authors": [
4          {
5              "name": "Christian"
6          }
7      ],
8      "require": {
9          "fakerphp/faker": "*"
10     }
11 }
12
```

```
EXPLORER
v REMOTO [SSH: 192.168.56.101]
> .git
> vendor
{ } composer.json
{ } composer.lock
@ README.md

{ } composer.lock U
1  {
2      "_readme": [
3          "This file locks the dependencies of your project to a known state",
4          "Read more about it at https://getcomposer.org/doc/01-basic-usage.md#installing-dependencies",
5          "This file is @generated automatically"
6      ],
7      "content-hash": "ca10ef466275bfd193922f6aa44a1e90",
8      "packages": [
9          {
10             "name": "fakerphp/faker",
11             "version": "v1.24.1",
12             "source": {
13                 "type": "git",
14                 "url": "https://github.com/FakerPHP/Faker.git",
15                 "reference": "e0ee18e6d6c3da3ce9fd97e5a0689a88a64b5"
16             },
17             "dist": {
18                 "type": "zip",
19                 "url": "https://api.github.com/repos/FakerPHP/Faker/zipball/e0ee18e6d6c3da3ce9fd97e5a0689a88a64b5",
20                 "reference": "e0ee18e6d6c3da3ce9fd97e5a0689a88a64b5",
21                 "shasum": ""
22             },
23             "require": {
24                 "php": "^7.4 || ^8.0",
25                 "psr/container": "^1.0 || ^2.0",
26                 "symfony/deprecation-contracts": "^2.2 || ^3.0"
27             },
28             "conflict": {
29                 "fzaninotto/faker": "*"
30             },
31             "require-dev": {
32                 "bamarni/composer-bin-plugin": "^1.4.1",
33                 "doctrine/persistence": "^1.3 || ^2.0",
34                 "ext-intl": "*",
35                 "phpunit/phpunit": "^9.5.26",
36                 "symfony/phpunit-bridge": "^5.4.16"
37             },
38             "suggest": {
39                 "doctrine/orm": "Required to use Faker\\ORM\\Doctrine",
40                 "ext-curl": "Required by Faker\\Provider\\Image to download images.",
41                 "ext-dom": "Required by Faker\\Provider\\HtmlLorem for generating random HTML.",
42                 "ext-iconv": "Required by Faker\\Provider\\ru_RU\\Text::realText() for generating real Russian text.",
43                 "ext-mbstring": "Required for multibyte Unicode string functionality."
44             },
45             "type": "library",

```

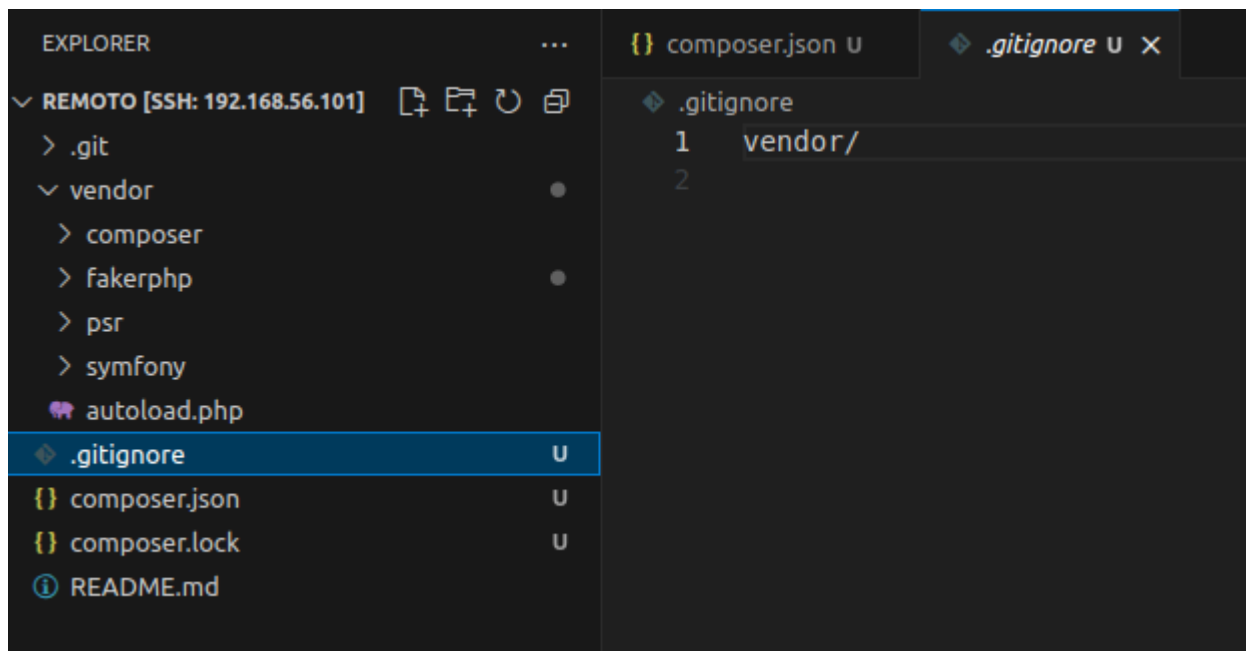
4. Realiza unha captura do contido do directorio `vendor`.

```
▼ vendor
> composer
> fakerphp
> psr
> symfony
🐘 autoload.php
{ } composer.json
{ } composer.lock
```

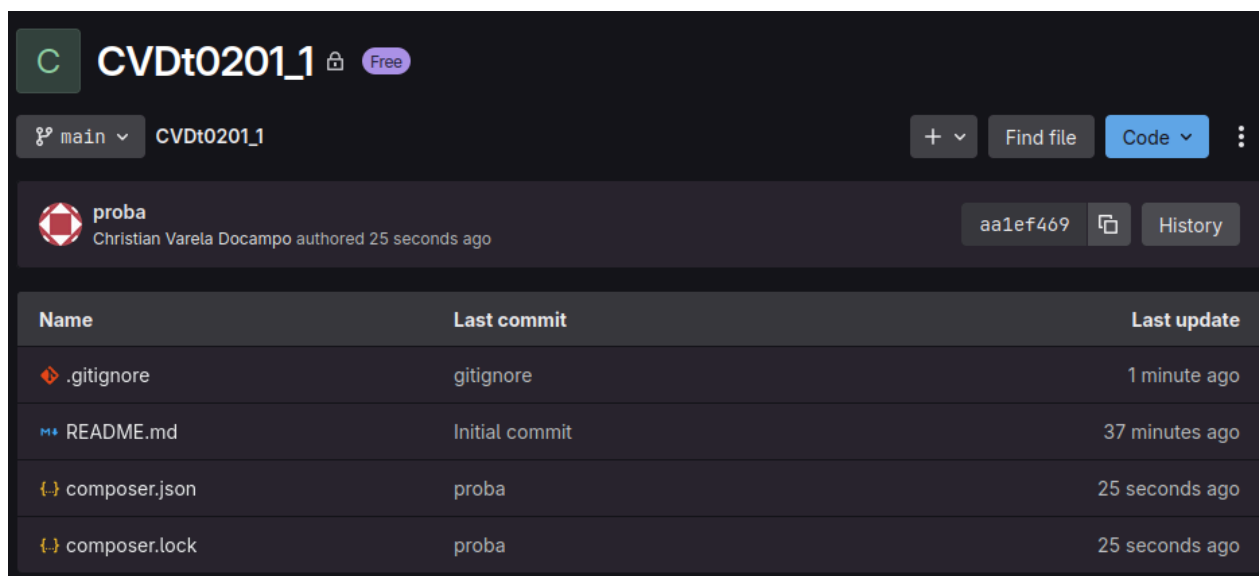
Non é recomendable subir ao repositorio o contido do directorio `vendor`. Estes directorio xeralmente ocupan moito espazo, e non nos interesa almacenalo no repositorio. Se queremos clonar o repositorio noutra localización, utilizando os ficheiros `composer.json` e `composer.lock` poderemos instalar de novo os paquetes necesarios para o correcto funcionamento do proxecto.

1. Crea o ficheiro `.gitignore` e engade o directorio `vendor` para que non se inclúa no repositorio.

Realiza unha captura do contido deste ficheiro.



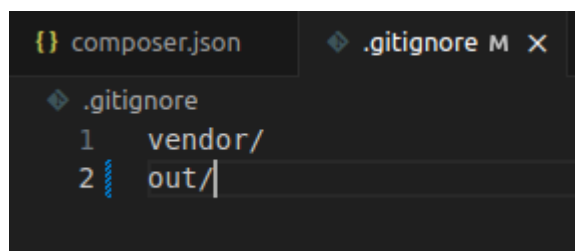
2. Antes de realizar o seguinte paso, asegúrate na extensión de Git de VSC que non está incluíndo o directorio `vendor`.
3. Realiza un `add`, un `commit` e un `push` para subir o repositorio a GitLab.com.
4. **Realiza unha captura** do repositorio en GitLab.com que se vexa o contido deste.



Agora comprobaremos como funciona o paquete instalado e a súa utilidade. En concreto, **o paquete `fakerphp/faker` xera datos aleatorios**. Isto é de gran utilidade para realizar probas. Por exemplo, o *script* de PHP que se axunta a continuación crea no directorio `out` un ficheiro CSV con datos xerados de nome, apelidos e datas de nacemento.

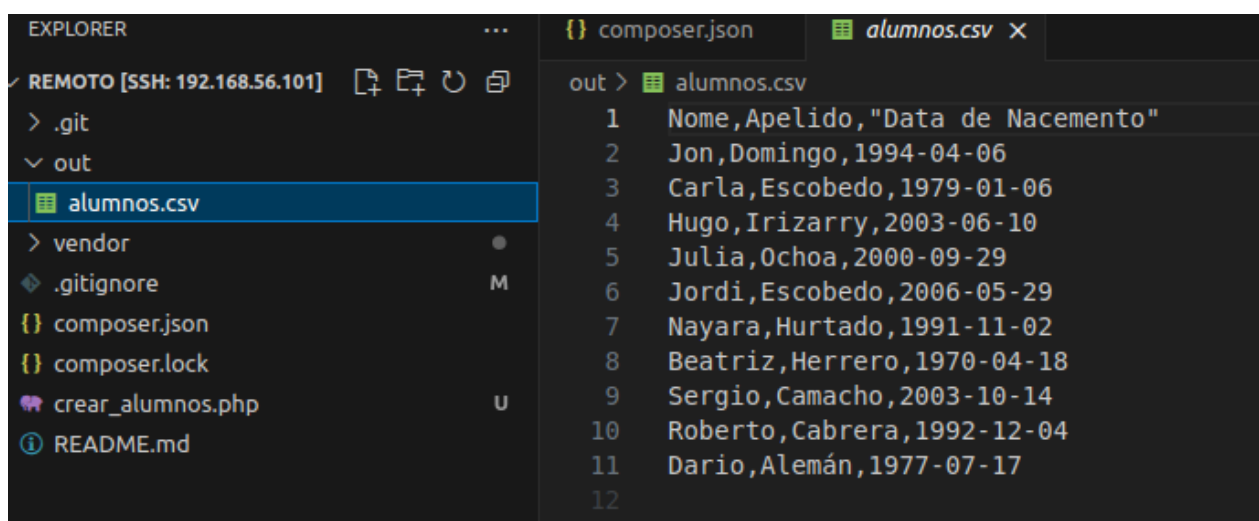
A continuación utilizáremolo e veremos como tamén que **os resultados de execución de *scripts* tampouco é recomendable subilo aos repositorios.**

1. Crea o ficheiro `xerar_alumnos.php` co contido indicado anteriormente.
2. Inclúe o directorio `out` no `.gitignore.` **Realiza capturas** do contido de `.gitignore.`



```
{} composer.json  .gitignore M X
.gitignore
1 vendor/
2 out/
```

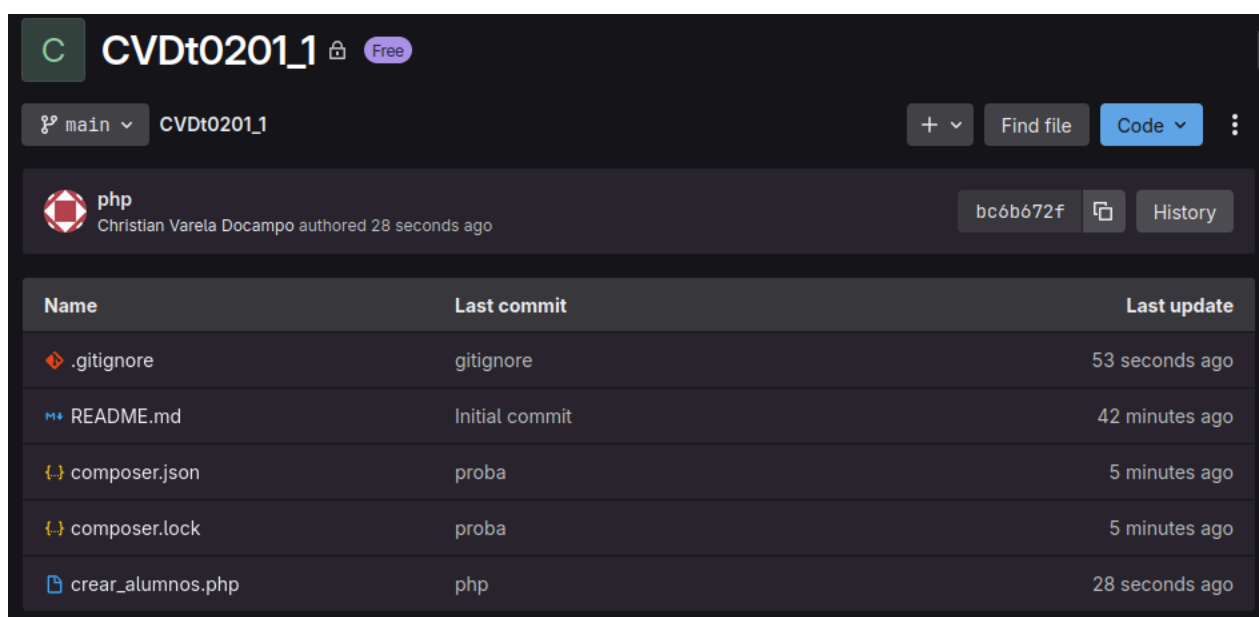
3. Executa o *script* co seguinte comando `php xerar_alumnos.php`. **Realiza unha captura** do ficheiro creado por este *script*.



```
EXPLORER
REMOTE [SSH: 192.168.56.101]
> .git
  out
    alumnos.csv
  vendor
  .gitignore M
  {} composer.json
  {} composer.lock
  crear_alumnos.php U
  README.md

out > alumnos.csv
1 Nome,Apellido,"Data de Nacemento"
2 Jon,Domingo,1994-04-06
3 Carla,Escobedo,1979-01-06
4 Hugo,Irizarry,2003-06-10
5 Julia,Ochoa,2000-09-29
6 Jordi,Escobedo,2006-05-29
7 Nayara,Hurtado,1991-11-02
8 Beatriz,Herrero,1970-04-18
9 Sergio,Camacho,2003-10-14
10 Roberto,Cabrera,1992-12-04
11 Dario,Alemán,1977-07-17
12
```

4. Sube o repositorio a GitLab.com.
5. **Realiza unha captura** do repositorio en GitLab.com que se vexa o contido deste.



CVDt0201_1 Free

main CVDt0201_1

php
Christian Varela Docampo authored 28 seconds ago

bc6b672f History

Name	Last commit	Last update
.gitignore	gitignore	53 seconds ago
README.md	Initial commit	42 minutes ago
composer.json	proba	5 minutes ago
composer.lock	proba	5 minutes ago
crear_alumnos.php	php	28 seconds ago

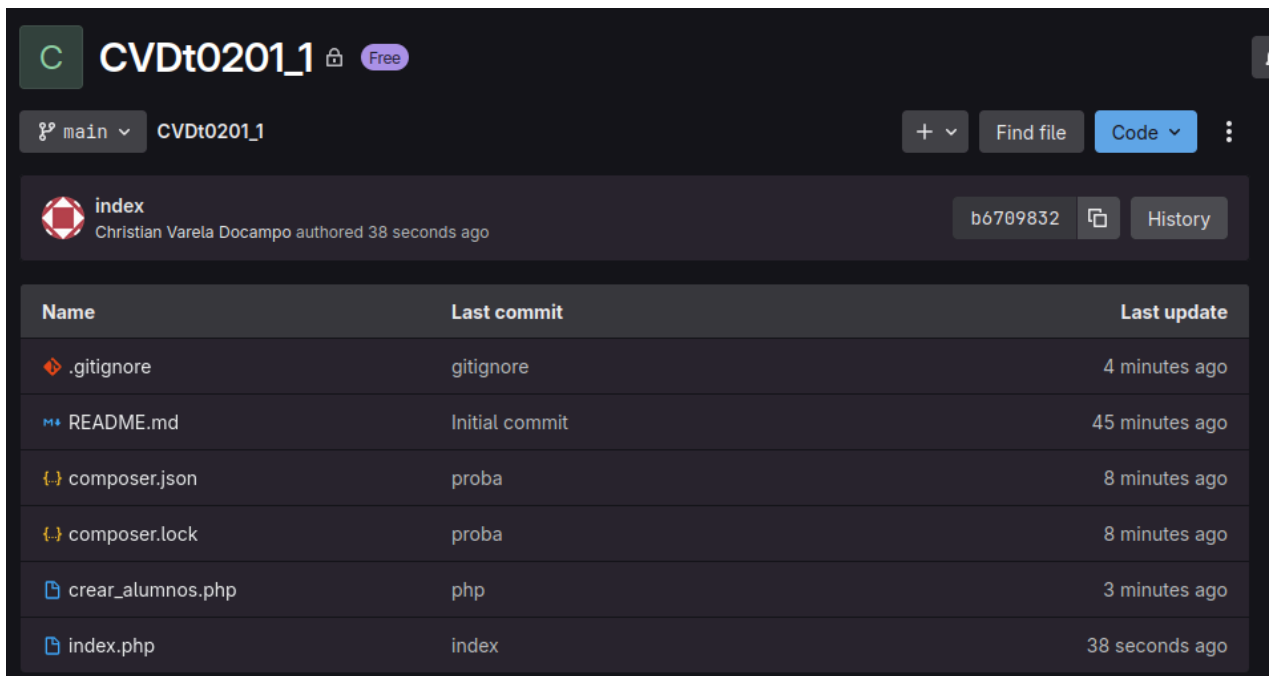
Por último probaremos o **servidor web de desenvolvemento** que trae incluído o propio intérprete de PHP. Este serviranos para facer probas rápidas de aplicación web realizadas nesta linguaxe de programación.

1. Crea o arquivo `index.php` co seguinte contido:

1. Executa o seguinte comando: `php -S 0.0.0.0:8000`. Este inicia o servidor PHP no porto 8000. A IP que se indica, é a rede dende onde os equipos poden acceder a este servidor (`0.0.0.0` é para calquera rede, deste xeito tamén podemos acceder dende o anfitrión).
2. No teu equipo, abre un navegador e escribe a seguinte URL: `IP_MÁQUINA_VIRTUAL:8000`. **Realiza capturas** da web que se mostra.



3. Cerra o servidor pulsando `Ctrl + C`.
4. Sube o repositorio a GitLab.com.
5. **Realiza unha captura** do repositorio en GitLab.com que se vexa o contido deste.



Exercicio 3: Xestor de paquetes NPM

Neste exercicio utilizaremos a máquina virtual anterior. Nun caso real, o **recomendable é utilizar outro máquina virtual** para non mesturar instalacións de PHP e Node. De feito, o máis recomendable é ter unha máquina diferente por proxecto. Comezaremos por instalar `Node.js`.

1. Pecha o VSC que está conectado a máquina virtual. Vólvetes conectar para comezar de novo.
2. Instala `Node.js` na máquina virtual.
3. **Realiza unha captura** coas versións de `Node.js` e `npm`.

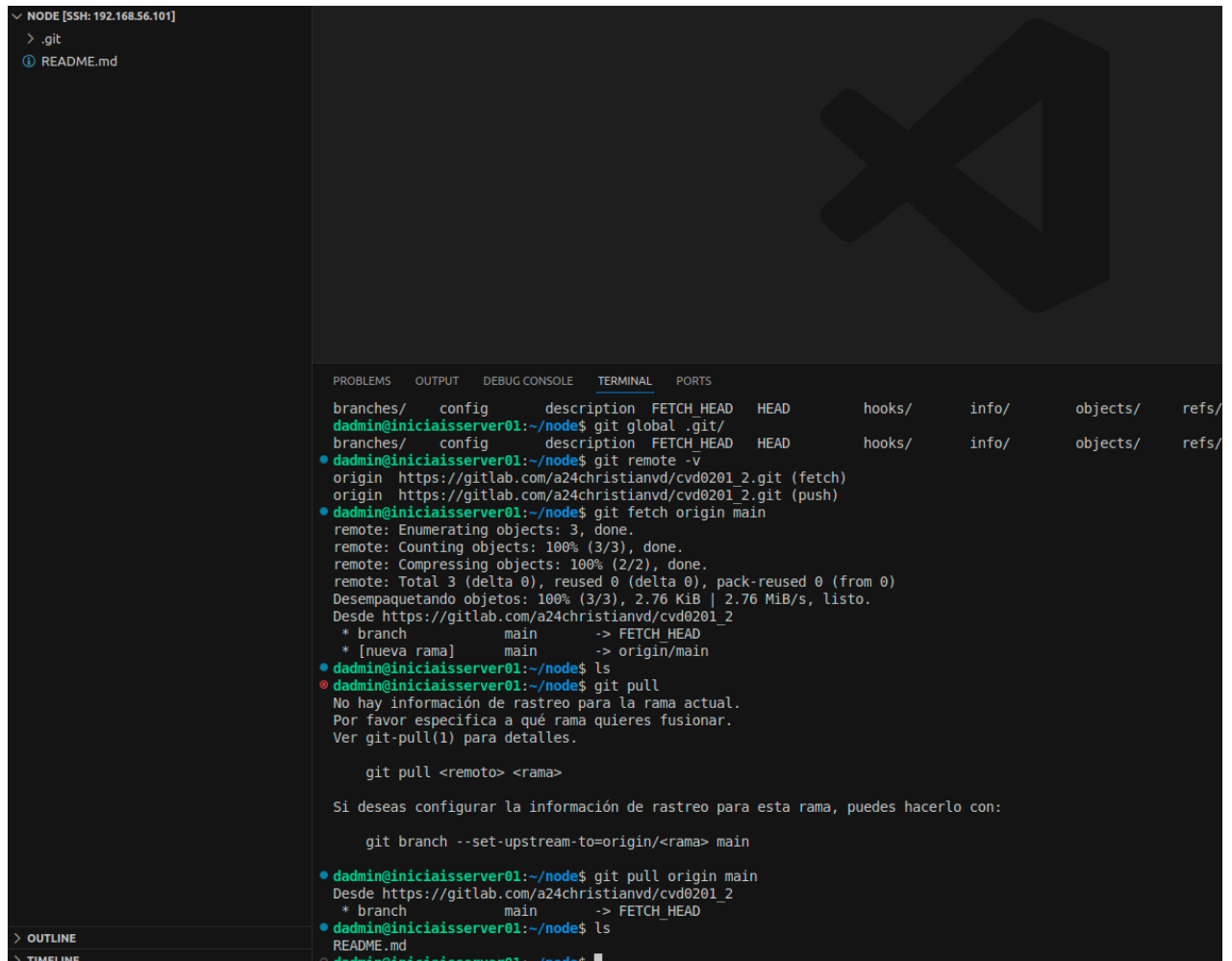
```
dadmin@iniciaisserver01:~/remoto$ node -v
v20.19.5
dadmin@iniciaisserver01:~/remoto$ npm -v
10.8.2
```


O que si deberemos e crear é un novo repositorio para realizar este novo proxecto con `Node.js`.

1. Crea un novo proxecto (en GitLab chámalle proxectos ao repositorios) de nome `iniciaist0201_2` con ficheiro `Readme.md`. **Realiza unha captura** da interface web co proxecto en GitLab.

olvideime de facer esta cando o repo de gitlab foi creado

2. Clona o repositorio na máquina virtual.
3. Preme na icona de ficheiros do menú da esquerda de Visual Studio Code. Preme en `Abrir Carpeta`. Selecciona o directorio onde clonaches o repositorio. Acepta tódalas mensaxes. **Realiza unha captura** da interface de VSC onde se vexan os ficheiros do repositorio e a terminal.



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows a file tree with `.git` and `README.md`. The main editor area is dark and contains a large, faint logo. At the bottom, the TERMINAL panel is active, displaying the following output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
branches/  config  description  FETCH HEAD  HEAD  hooks/  info/  objects/  refs/
dadmin@iniciaisserver01:~/node$ git global .git/
branches/  config  description  FETCH HEAD  HEAD  hooks/  info/  objects/  refs/
• dadmin@iniciaisserver01:~/node$ git remote -v
origin  https://gitlab.com/a24christianvd/cvd0201_2.git (fetch)
origin  https://gitlab.com/a24christianvd/cvd0201_2.git (push)
• dadmin@iniciaisserver01:~/node$ git fetch origin main
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Desempaquetando objetos: 100% (3/3), 2.76 KiB | 2.76 MiB/s, listo.
Desde https://gitlab.com/a24christianvd/cvd0201_2
* branch          main      -> FETCH HEAD
* [nueva rama]     main      -> origin/main
• dadmin@iniciaisserver01:~/node$ ls
• dadmin@iniciaisserver01:~/node$ git pull
No hay información de rastreo para la rama actual.
Por favor especifica a qué rama quieres fusionar.
Ver git-pull(1) para detalles.

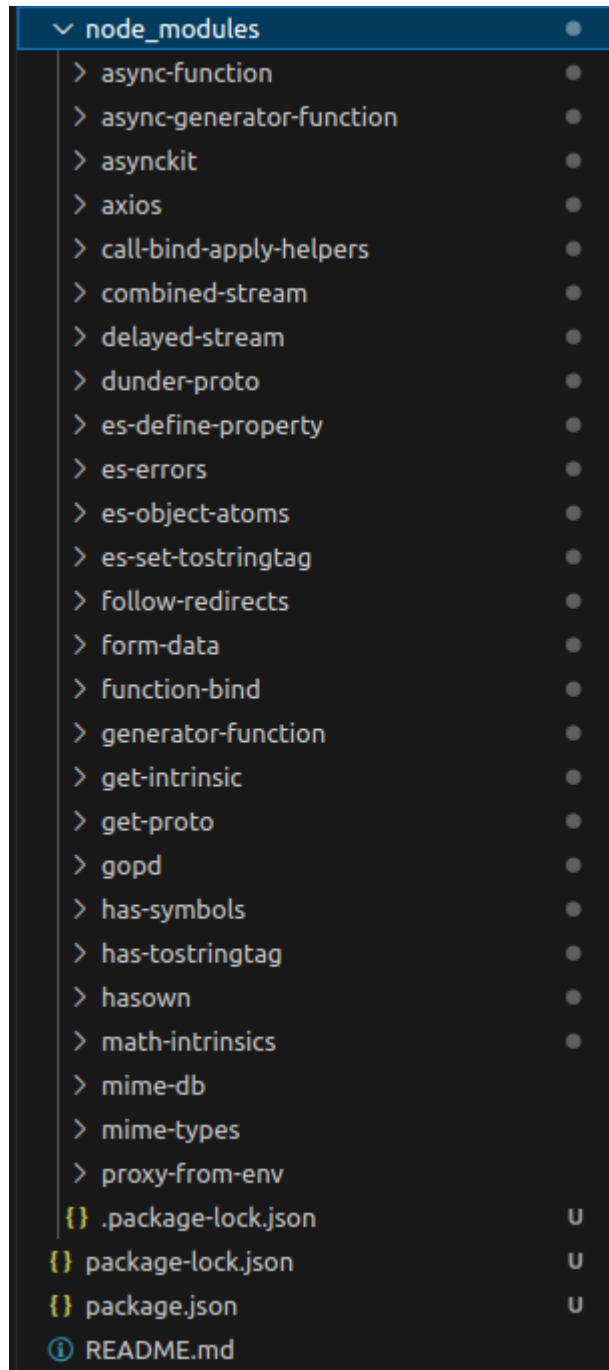
git pull <remoto> <rama>

Si deseas configurar la información de rastreo para esta rama, puedes hacerlo con:

git branch --set-upstream-to=origin/<rama> main
• dadmin@iniciaisserver01:~/node$ git pull origin main
Desde https://gitlab.com/a24christianvd/cvd0201_2
* branch          main      -> FETCH HEAD
• dadmin@iniciaisserver01:~/node$ ls
README.md
• dadmin@iniciaisserver01:~/node$
```

Agora imos **crear o novo proxecto de `Node.js` utilizando `npm`**. Tamén instalaremos algún paquete e configuraremos correctamente o repositorio ao igual que fixemos no exercicio anterior.

1. Crea con `npm init` o ficheiro `package.json`. Non tes porque contestar a ningunha cuestión. Tan só o final tes que indicar que si que queres que se cree o ficheiro `package.json`.
2. **Realiza capturas** do contido do ficheiro `package.json`.



6. Inclúe o directorio `node_modules` no `.gitignore`.
7. Realiza un `commit` e un `push` para subir o repositorio a GitLab.com.
8. **Realiza unha captura** do repositorio en GitLab.com que se vexa o contido deste.

CVD0201_2

main
cvd0201_2
+
Find file
Code

node
Christian Varela Docampo authored 49 seconds ago
796126f2
History

Name	Last commit	Last update
.gitignore	gitignore	1 minute ago
README.md	Initial commit	9 minutes ago
package-lock.json	node	49 seconds ago
package.json	node	49 seconds ago

Axios é unha librería que nos permite realizar chamadas HTTP de xeito sinxelo. Utilízase moito para consumir API-REST. No seguinte *script* de `Node.js` facemos unha proba do funcionamento desta librería. Neste caso utilizaremos a API-REST de [OpenMeteo](#) que nos permite obter datos meteorolóxicos. O seguinte *script* fai unha petición datos meterolóxicos para Santiago de Compostela.

1. Crea un ficheiro `index.js` co contido do ficheiro anterior.
2. Executa o *script* anterior co comando `node index.js`. **Realiza capturas** da saída da execución do *script*.

```

dadmin@iniciaisserver01:~/node$ node index.js
🌍 Cidade: Santiago de Compostela
🌡️ Temperatura: 18.2 °C
💨 Vento: 3.5 km/h
🌀 Dirección do vento: 204°
🕒 Hora da medición: 2025-10-03T12:45

```

3. Realiza un `commit` e un `push` para subir o repositorio a GitLab.com.
4. **Realiza unha captura** do repositorio en GitLab.com que se vexa o contido deste.

CVD0201_2

main
cvd0201_2
+
Find file
Code

index
Christian Varela Docampo authored 1 minute ago
0a2ccc01
History

Name	Last commit	Last update
.gitignore	gitignore	2 days ago
README.md	Initial commit	2 days ago
index.js	index	1 minute ago
package-lock.json	node	2 days ago
package.json	node	2 days ago